

Exercício Prático: API de Pedidos da Cantina Lions

Turma: LionsDev

Tópicos: APIs REST com Express.js, Integração com MongoDB (Mongoose), Schemas e Modelos, Validações básicas com Mongoose, Operações CRUD assíncronas (async/await), Query Params (filtros), Regras de Negócio simples e Status Codes HTTP.

1. Contexto

A **Cantina Lions** recebe pedidos de alunos durante os intervalos, mas ainda anota tudo em papel. Isso causa confusão no valor dos pedidos, nos pagamentos e no acompanhamento do que ainda está pendente.

Você deverá criar uma API REST simples conectada ao MongoDB para gerenciar pedidos da cantina. A estrutura deve seguir o modelo usado em aula: `server.js`, `db.js` e `models/`, com as rotas escritas diretamente no `server.js`.

2. Configuração Inicial

Crie uma API Node.js com Express e Mongoose.

Requisitos estruturais:

- Crie um arquivo `.env` contendo as variáveis `MONGO_URI` e `PORT` (porta `3000`).
- Crie a pasta `src`.
- Crie o arquivo `src/db.js` para gerenciar a conexão com o MongoDB.
- Crie o arquivo `src/server.js` como ponto de entrada da aplicação.
- Crie a pasta `src/models` e nela o arquivo `pedido.js`.
- Todas as rotas devem ficar no `src/server.js`, como fizemos em sala.

O Modelo (Schema) do Pedido

O Schema do Mongoose para os `pedidos` deve conter os seguintes campos:

- `nomeCliente`: Tipo `String`, obrigatório.
- `item`: Tipo `String`, obrigatório (deve aceitar apenas: `Salgado`, `Suco`, `Combo` ou `Bolo`).
- `quantidade`: Tipo `Number`, obrigatório.
- `formaPagamento`: Tipo `String`, obrigatório (deve aceitar apenas: `Dinheiro`, `Pix` ou `Cartão`).

- **observacao** : Tipo **String** , opcional.
 - **valorUnitario** : Tipo **Number** (será calculado automaticamente pela API).
 - **valorTotal** : Tipo **Number** (será calculado automaticamente pela API).
 - **status** : Tipo **String** , com valor padrão de **"Pendente"** (deve aceitar apenas: **Pendente** , **Pago** ou **Entregue**).
-

3. Requisitos Obrigatórios e Regras de Negócio

3.1 Cadastrar Pedido (CREATE)

Crie a rota **POST /pedidos** .

O corpo da requisição enviará os dados do pedido (exceto **valorUnitario** , **valorTotal** e **status**).

```
{
  "nomeCliente": "João Pedro",
  "item": "Combo",
  "quantidade": 3,
  "formaPagamento": "Pix",
  "observacao": "Sem maionese"
}
```

Regras:

1. **Definição do Valor Unitário:** Antes de salvar no banco, o backend deve preencher **valorUnitario** seguindo esta tabela:
 - **Salgado** : R\$ 8
 - **Suco** : R\$ 6
 - **Combo** : R\$ 12
 - **Bolo** : R\$ 5
2. **Cálculo do Valor Total:** Multiplique **valorUnitario** por **quantidade** .
3. **Desconto por Quantidade:** Se a quantidade for maior ou igual a 5, aplique 10% de desconto sobre o valor total.
4. **Retorno:** Salve o pedido no banco e retorne o documento criado com status **201** .

3.2 Listar Todos os Pedidos (READ)

Crie a rota **GET /pedidos** .

Regras:

- A rota deve retornar todos os pedidos salvos no MongoDB usando `Pedido.find()`.
- Retorne o array de resultados com status `200`.

3.3 Buscar Pedidos por Cliente (QUERY PARAMS)

Crie a rota `GET /pedidos/busca`.

Esta rota deve aceitar um filtro opcional via Query Params chamado `cliente`:

- Exemplo de URL: `http://localhost:3000/pedidos/busca?cliente=joao`
- A busca deve retornar todos os pedidos em que o nome do cliente contenha o texto pesquisado.

3.4 Atualizar Status do Pedido (UPDATE)

Crie a rota `PATCH /pedidos/:id`.

O corpo da requisição deve enviar apenas o novo status (ex: `{ "status": "Pago" }`).

Regras:

- Busque o pedido pelo ID e atualize o status usando `findByIdAndUpdate`.
- Se o ID não for encontrado, responda com status `404`.
- Retorne o documento atualizado com status `200`.

3.5 Remover Pedido (DELETE)

Crie a rota `DELETE /pedidos/:id`.

Regras:

- Remova o pedido do banco de dados pelo ID usando `findByIdAndDelete`.
- Se não encontrar o registro, retorne status `404`.
- Se remover com sucesso, retorne status `200` com uma mensagem de confirmação.

4. Testes Esperados

Realize testes na sua API seguindo este fluxo de validação:

1. Cadastre um pedido de 2 `Salgado` e verifique se o total ficou R\$ 16.
2. Cadastre um pedido de 5 `Combo` e verifique se o desconto de 10% foi aplicado.
3. Liste todos os pedidos cadastrados.
4. Faça uma busca pelo nome de um cliente.
5. Atualize o status de um pedido para `Pago`.
6. Delete um dos pedidos informando o ID.

5. Dicas para a Implementação

- Use `if` ou `switch` para definir o preço do item.
 - Para aplicar desconto, multiplique o total por `0.9`.
 - No `findByIdAndUpdate`, passe `{ new: true, runValidators: true }`.
 - Lembre-se de usar `app.use(express.json())` no `src/server.js` para conseguir ler o corpo das requisições.
-

LionsDev • Professor Nicolas Cardoso Motta

Exercício Prático de Mongoose e MongoDB - Módulo 08